

ПОЛИМОРФИЗЪМ

От предния час запомнихме, че ако даден метод в родителския клас от самото създаване е предназначен за промяна, той е `virtual`, а наследникът го модифицира чрез `override`. Когато искаме наследниците на наследника вече да не могат да променят по-нататъчни `override`, ги забраняваме чрез `sealed`, като е добра практика винаги да се добавя коментар защо се спира презаписването. В логиката не е желателно да има много `sealed`, а ако има, значи трябва да я променим. Ако даден метод в родителя от създаването си не е предназначен за промяна с `virtual`, но все пак се наложи някой от наследниците да използва своя адаптация за него, родителският метод се скрива чрез `new` в наследника. Това не е нов метод, а просто скриване на родителския.

Полиморфизмът е една от основните концепции в ООП, заедно с:

- Наследяване
- Капсулация
- Абстракция

C# предоставя два основни вида полиморфизъм:

1) Статичен (компиляционен) полиморфизъм - чрез **method overloading** (претоварване на методи) като задачата за Area, която правехме, а кой точно от тези методи Area да се изпълни, се определя от броя, последователността и типа на input variables. Компиляторът автоматично ще познае правилната „формула“ и ще изпълни тялото на метода.

2) Динамичен (runtime) полиморфизъм - чрез **virtual/override** и interfaces.

Пример за статичен полиморфизъм:

```
class Calculator
{
    1 reference
    public int Add(int a, int b)
    {
        return a + b;
    }

    1 reference
    public double Add(double a, double b)
    {
        return a + b;
    }

    1 reference
    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}
```

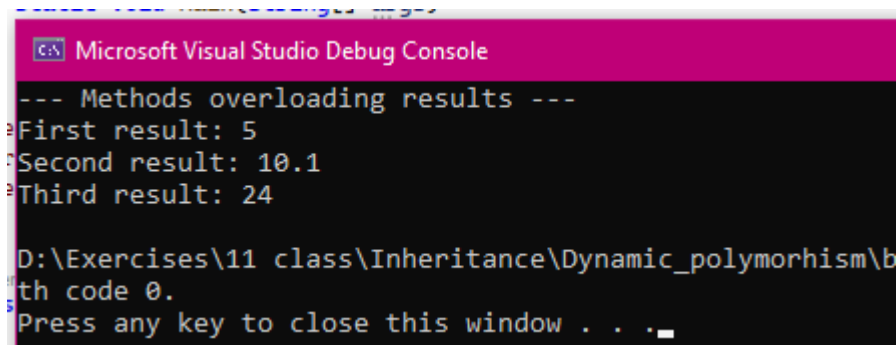
Компиляторът автоматично намира правилния метод (независимо от последователността на написването на методите, а само от съвпадението на подадените входни променливи):

```

static void Main(string[] args)
{
    Calculator c = new Calculator();
    Console.WriteLine($"--- Methods overloading results ---");
    First result: {c.Add(2, 3)}
    Second result: {c.Add(4.5, 5.6)}
    Third result: {c.Add(7, 8, 9)}");
}

```

Резултат:



```

Microsoft Visual Studio Debug Console
--- Methods overloading results ---
First result: 5
Second result: 10.1
Third result: 24

D:\Exercises\11 class\Inheritance\Dynamic_polymorphism\b
th code 0.
Press any key to close this window . . .

```

Пример за динамичен полиморфизъм – при родителя, още от създаването на даден метод, ние знаем, че неговата логика ще подлежи на промяна от наследниците, затова родителския метод става `virtual`, а ако наследниците искат да го презапишат, заявяват това с `override`. В случая упражняваме вградения виртуален метод `ToString()`, който не присъства в нашия клас `Calculator`, но ние използваме и презаписваме. Неговата роля е да подреди (форматира) текст, връща тип `string`, който после може да използваме за различни цели. В случая, `ToString()` се използва вместо `DisplayInfo()`, с който сме свикнали и директно отпечатва съобщение на конзолата. `ToString()` е текст, който после може да бъде вложен в `Main()` като променлива или директно в `Console.WriteLine()`. `DisplayInfo()` съдържаше `CW` и затова, ако го вложехме в друг `CW`, даваше грешка (едно отпечатване в друго). `ToString()`, обаче е текст, който спокойно може да бъде даден в `CW`.

```

class Calculator
{
    3 references
    protected internal string name;
    1 reference
    *1 public Calculator() // ctor + tab + tab
    {
    }
    1 reference
    *2 public Calculator(string name) // конструира нов обект
    {
        this.name = name; заради малките букви
    }
    1 reference
    public override string ToString() вграден метод за презаписване
    {
        return $"This is {name}";
    }
    1 reference
    1) public int Add(int a, int b)
    {
        return a + b;
    }
    1 reference
    2) public double Add(double a, double b)
    {
        return a + b;
    }
    1 reference
    3) public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}

```

В нашия клас имаме 2 конструктора (1 празен и 1, който приема текст) и 3 метода с еднакви имена. Във втория обект, който създаваме c1, макар да сме подали текст за обработка, той да е извикан и създаден от втория конструктор. Интересното е, че променливата name е защитена и с малка буква, заради което я използваме с this., но също така, ние може да я достъпим и променим с директно извикване, без да минаваме през конструктор.

```

static void Main(string[] args)
{
    Calculator c = new Calculator(); празен ctor
    Console.WriteLine($"--- Methods overloading results ---");
    First result: {c.Add(2, 3)}
    Second result: {c.Add(4.5, 5.6)} няма двусмислие
    Third result: {c.Add(7, 8, 9)}";

    Console.WriteLine($"--- Static polymorphism ---");
    Calculator c1 = new Calculator("input value"); другия ctor
    c1.name = "changed value"; достъпва и променя
    Console.WriteLine(c1.ToString());
    подрежда текст
}

```

```
Select Microsoft Visual Studio Debug Console
--- Methods overloading results ---
First result: 5
Second result: 10.1
Third result: 24
--- Static polymorphism ---
This is changed value
```

След като се намираме в един клас, ние може да създаваме обекти чрез всеки от неговите конструктори, също така да достъпваме променливи, да ги променяме и използваме предложените методи в него.

Като упражнение – създайте 1 нов обект от калкулатора, който първоначално е празен, но после му задавате име, извиквате метода за събиране на десетични числа и отпечатвате.

Примерно решение:

```
Console.WriteLine($"--- Testing polymorphism ---");
Calculator c3 = new Calculator();
c3.name = "first ctor with included name";
Console.WriteLine($"One of methods: {c3.Add(12.34, 56.78)}");
Console.WriteLine(c3.ToString());
```

```
--- Testing polymorphism ---
One of methods: 69.12
This is first ctor with included name
```